

OE-EC604B — Operating System

Semester VI • 3L:0T:0P • 3 Credits

Module 2 Exam Notes: Operating System Structures

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	System Components and Operating System Services	2
1.1	Operating System Services	2
1.2	System Calls	2
2	Operating System Architecture Structures	3
2.1	Monolithic Structure	3
2.2	Layered Structure	3
2.3	Microkernel Structure	3
2.4	Hybrid Systems	4

1. System Components and Operating System Services

An operating system provides an environment for the execution of programs by presenting services to both users and system processes.

1.1 Operating System Services

The OS provides services that make computer usage convenient and efficient:

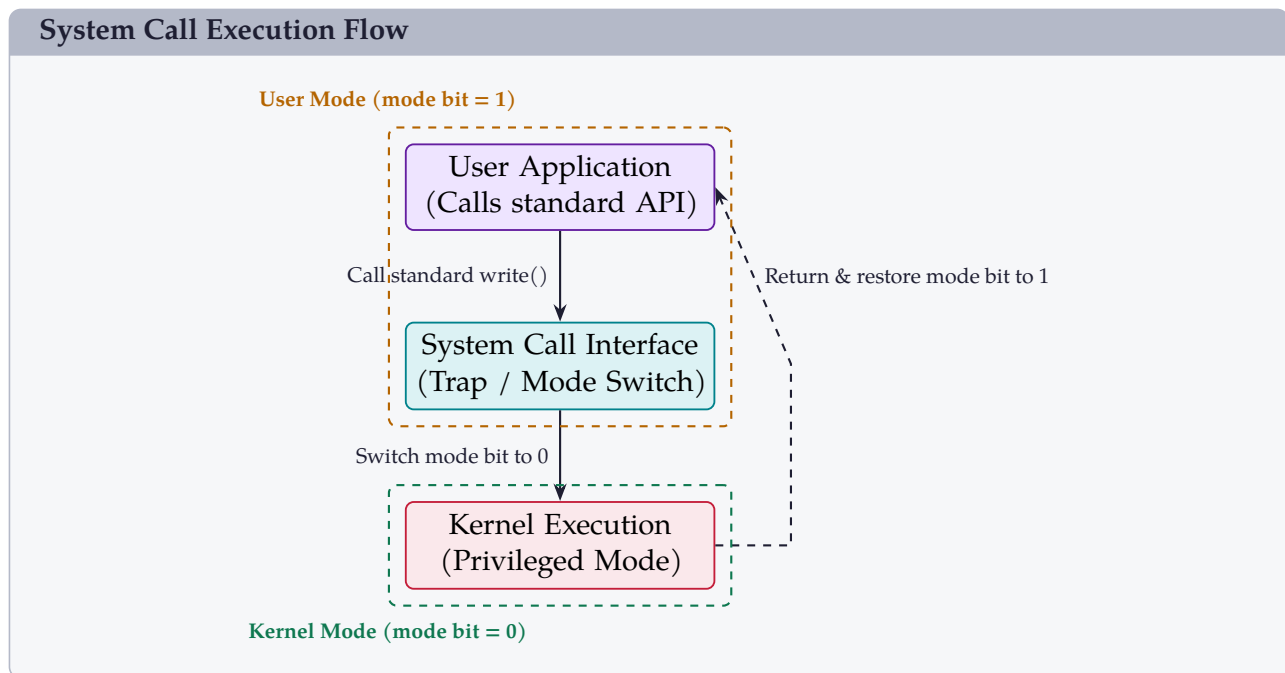
1. **User Interface (UI):** Command Line Interface (CLI), Graphics User Interface (GUI), or Batch Interface.
2. **Program Execution:** Loading a program into memory, executing it, and terminating it (either normally or abnormally).
3. **I/O Operations:** Managing read/write transfers to files or hardware devices since user programs cannot access hardware directly.
4. **File-System Manipulation:** Creating, deleting, searching, reading, writing, and controlling file access permissions.
5. **Communications:** Exchanging information between processes running on the same computer or different machines via networks (using shared memory or message passing).
6. **Error Detection:** Continuously monitoring for hardware errors (memory parity, power failure) or software errors (arithmetic overflow, illegal memory access) and executing handling procedures.
7. **Resource Allocation:** Distributing CPU cycles, main memory, and file storage dynamically among active users or jobs.
8. **Accounting:** Tracking user resource usage for billing, statistics, or system profiling.
9. **Protection & Security:** Controlling access to system resources and securing user authentication.

1.2 System Calls

Definition: System Call

A **System Call** is the programmatic interface that allows a running program (in user space) to request services directly from the operating system kernel (in kernel space).

- **Dual-Mode Operation:** Modern hardware supports at least two modes of operation: **User Mode (mode bit = 1)** and **Kernel/Supervisor Mode (mode bit = 0)**. System calls trigger a software interrupt (trap instruction) that shifts execution from user mode to kernel mode to run privileged instructions.



2. Operating System Architecture Structures

Operating systems must be structured to ensure reliability, performance, and ease of modification. There are four traditional design methodologies:

2.1 Monolithic Structure

In a monolithic operating system, all OS code (scheduling, filesystem, memory management, drivers) runs in a single address space in kernel mode.

- **Advantages:** High performance due to minimal context switching overhead.
- **Disadvantages:** Lack of safety — a bug in any module (like a camera driver) can crash the entire system.

2.2 Layered Structure

The OS is broken into a hierarchy of layers (0 to N). Layer 0 is the hardware; Layer N is the user interface.

- **Design Rule:** Each layer is built upon and can only invoke services of lower-level layers.
- **Advantages:** Modular development, simplified debugging.
- **Disadvantages:** Hard to define layers cleanly, and poor performance because each layer crossing introduces execution overhead.

2.3 Microkernel Structure

The microkernel architecture structures the OS by removing all non-essential components from the kernel, implementing them instead as user-space programs (servers).

- **Core Kernel Services:** Low-level memory management, thread scheduling, and Inter-Process Communication (IPC).

- **Advantages:** High security and reliability. If a filesystem service crashes, the rest of the OS remains unaffected.
- **Disadvantages:** High performance overhead because client programs must communicate with user-space servers via message passing, which requires frequent context switches.

2.4 Hybrid Systems

Most modern operating systems (e.g., Windows NT, macOS, Linux) use a hybrid design. They combine monolithic speed with microkernel modularity, running core modules in a single address space while using dynamic modules (loadable kernel modules) for extensibility.

Quick Revision — Module 2

Operating System Structures | OE-EC604B

System Call Parameter Passing Methods

Register Method	Parameters passed directly in CPU registers. Fastest, but limited parameter capacity.
Block / Table Method	Parameters written to a block in memory; pointer to the block is passed in a register. Unlimited size.
Stack Method	Parameters pushed onto the user stack and popped off by the kernel. Unlimited size.

One-Line Definitions

- **System Call:** Programmatic request by user-mode applications to execute kernel-mode services.
- **Mode Bit:** A hardware register flag indicating User Mode (1) or Kernel Mode (0).
- **Monolithic Kernel:** OS design where all services run in a single address space inside kernel mode.
- **Microkernel:** Minimalist kernel executing only IPC, scheduling, and basic memory management.
- **Layered Architecture:** Hierarchy of OS layers where layer i relies only on services of layer $i - 1$.
- **Trap:** A software-generated interrupt caused by an error or system call request.
- **API (Application Programming Interface):** Standard set of functions wrapper around system calls (e.g., Win32, POSIX).
- **System Program:** File utilities, compilers, or editors that provide a shell wrapper around system calls.

Frequently Asked / Viva Questions

Q1. Why is dual-mode operation supported by hardware in modern operating systems?

To protect the system from malicious or poorly written user applications. Privileged instructions (like I/O control or memory protection access) can only be executed in Kernel Mode.

Q2. Describe the sequence of actions that occur when a system call is made.

The user program invokes the API function → API puts system call number in a register → triggers software interrupt (trap) → hardware switches mode bit to 0 → execution jumps to system call handler in kernel → handler executes kernel code → mode bit is reset to 1 → control returns to user program.

Frequently Asked / Viva Questions (contd.)**Q3. Compare Monolithic Kernels and Microkernels in terms of reliability and performance.**

Monolithic: High performance (fast function calls, no IPC overhead) but low reliability (any driver crash can bring down the entire OS). Microkernel: High reliability (isolated user-space services) but slower performance (heavy IPC message passing and context switches).

Q4. What are system programs? Give two examples.

System programs are user-space utilities that provide a convenient environment for program development and execution. Examples: file command programs (`cp`, `mv`) and status utilities (`ps`, `top`).

Q5. What are the three common methods used to pass parameters to the operating system during a system call?

1. Pass parameters in CPU registers. 2. Store parameters in a memory block/table and pass the block address in a register. 3. Push parameters onto the system stack.

Q6. What is the difference between a system call and a library function?

A system call is handled directly by the kernel and involves context switching and privilege escalation. A library function (like `strlen()`) executes entirely in user mode without kernel intervention.

Q7. Explain the concept of device drivers in OS structure.

Device drivers are specialized software modules that translate general I/O commands from the OS kernel into device-specific commands that can be interpreted by hardware controllers.

Q8. Why does the layered architecture model suffer from performance issues?

Each layer boundary requires crossing and parameter validation. An application requesting a file read might have to traverse multiple layers sequentially, incurring cumulative overhead.

Q9. What is the POSIX standard?

Portable Operating System Interface; a family of standards specified by the IEEE to clarify API compatibility between UNIX-like operating systems.

Q10. How does a hybrid operating system achieve both reliability and speed?

By using a monolithic core for speed (running speed-critical elements like graphics and file systems in kernel space) but supporting dynamic loadable kernel modules (LKMs) for driver modularity.

Comparison of OS Architectures

Architecture	Context Switch Cost	Reliability	Extensibility
Monolithic	Extremely Low (no IPC)	Poor (no protection)	Hard (requires recompile)
Layered	High (traverses layers)	Moderate	Moderate
Microkernel	Very High (heavy IPC)	Excellent	Very Easy (add user servers)